

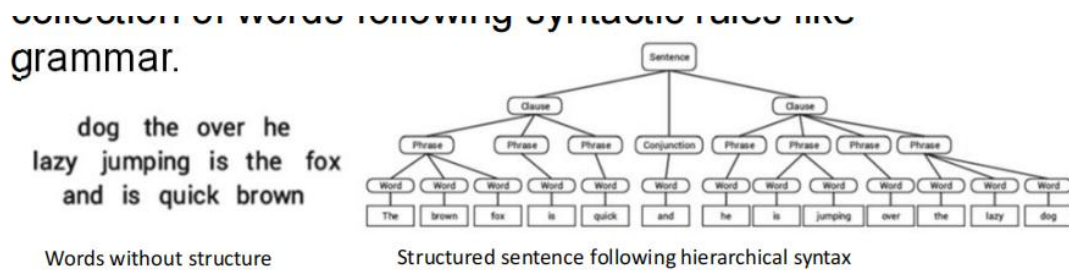
自然语言处理

1.1 自然语言&NLP

在英语中，词汇组合形成更大单位，如短语、从句和句子。

句子是一种有结构的表达方式，遵循语法规则。

图示：无结构的单词 → 有语法的句子



src: Dipanjan Text Analytics with Python, 2016

自然语言：人类自然发展出来的语言（与计算机编程语言相对）。示例：英语、中文。

NLP 目标：让计算机理解人类语言。

难点：自然语言是非结构化的。

1.2 NLP 框架

【NLP 框架】

自然语言处理的两个核心部分：

- 自然语言理解 (NLU)：分析语言的语法和语义，把输入映射到内部表示（把输入转为机器可理解的表示形式）
- 自然语言生成 (NLG)：根据数据生成自然语言文本或句子（生成自然语言的有意义文本）

常见处理流程：

- 词汇分析 (Lexical analysis)
- 句法分析 (Syntactic analysis)
- 语义分析 (Semantic analysis)
- 篇章整合 (Discourse integration)
- 语用分析 (Pragmatic analysis)

【Python】-NLTK (Natural Language Toolkit)

NLTK 是一个 Python 的自然语言处理模块，提供语料库、词典和文本处理能力。

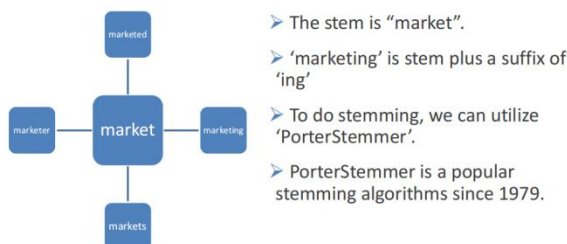
语料 (Corpus) 指存储在计算机中的大量文本，是语言学分析的基础。

示例语料库包括：BNC（英国国家语料库）、COCA（当代美国英语语料库）、Brown 语料库等。

文本预处理操作：

- 去掉多余空格
- 统一大小写
- 分词 (Tokenization)：将文本分割成单词
- 词干提取 (Stemming)
- 词形还原 (Lemmatization)

Stemming



【文本预处理操作】

- **分词 (Tokenization):**
 - 用分隔符（如空格、标点）将文本切割成单词或句子。
 - 示例：使用 PunktSentenceTokenizer 拆分句子。
- **词干提取 (Stemming):**
 - 英文单词常有后缀 (-s, -ed, -ing 等)。
 - 词干 (Stem) 是去掉后缀的基础形式。
 - 示例：market → marketed / marketing → stem = "market"。
 - 工具：PorterStemmer（经典算法，1979）。
- **词形还原 (Lemmatization)**
 - 与词干提取类似，但会得到有效的词根 (lemma)。
 - 工具：WordNetLemmatizer。
 - 示例：
 - Beaten → beat
 - Happiest → happy
- **词性标注 (POS Tagging)**
 - POS (Part-of-Speech) 标注：把句子中的每个单词标注为名词、动词、形容词等。
 - 工具：pos_tag()。
 - 示例：
 - "Everyday" 被标注为 NNP（专有名词）。
 - 其他标记如：VBZ（动词），DT（限定词），JJ（形容词）

1.3 【文本分类器】

● 文本向量化

- 向量化 (Vectorization): 把文本转为数值向量。
- TF-IDF: 衡量单词在文本中的重要性。
- TF = 词频
- IDF = 逆文档频率
- $TF\text{-}IDF = TF \times IDF$

● 文本分类:

- 训练 (Training): 用标注数据训练模型。
- 预测 (Prediction): 对新数据进行分类。
- 示例任务: 真假信息分类 (Nonsense / No Nonsense), 用 RandomForestClassifier 训练分类器

Vectorizing Text

- Turn text into vectors.
- TF-IDF: Term Frequency Inverse Document Frequency.
 - Numerical statistics to reflect how important a word is to a document in a collection of text.
 - TF: The frequency of a term occurrence.
 - E.g. 'apple' appears 5 times in document 1, total number of terms in document 1 is 100. The TF is $5/100=0.05$.
 - IDF: Measures the importance of the term in the entire corpus.
 - E.g. 'apple' appears 8 out of the 10 documents. IDF value is $\ln(10/8) = 0.223$
 - $TF\text{-}IDF = TF \times IDF = 0.05 \times 0.223 = 0.01115$

TF-IDF

• Text A: Jupiter is the largest planet

• Text B: Mars is the fourth planet from the sun

The table below shows the values of TF for A and B, IDF, and TFIDF for A and B.

Words	TF (A)	TF (B)	IDF	TFIDF (A)	TFIDF (B)
Jupiter	1/5	0	$\ln(2/1)=0.69$	0.138	0
in	1/5	1/8	$\ln(2/2)=0$	0	0
the	1/5	2/8	$\ln(2/2)=0$	0	0
largest	1/5	0	$\ln(2/1)=0.69$	0.138	0
planet	1/5	1/8	$\ln(2/2)=0$	0.138	0
Mars	0	1/8	$\ln(2/1)=0.69$	0	0.086
fourth	0	1/8	$\ln(2/1)=0.69$	0	0.086
from	0	1/8	$\ln(2/1)=0.69$	0	0.086
sun	0	1/8	$\ln(2/1)=0.69$	0	0.086

Illustration extracted from: KDnuggets. See further details

<https://www.kdnuggets.com/2022/09/convert-text-documents-tfidf-matrix-tfidfvectorizer.html>

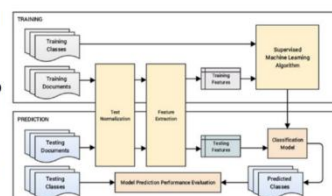
Vectorizing Text (文本向量化)

- Turn text into vectors.
 - 将文本转换为向量。
- TF-IDF: Term Frequency Inverse Document Frequency (词频-逆文档频率)
 - 一种数值统计方法, 用来衡量一个词在一篇文章中相对于整个语料库的重要程度。
 - TF (Term Frequency) 词频:
 - 表示某个词在当前文档中出现的频率。
 - 例: 词 'apple' 在文档1中出现5次, 而文档1的总词数是100,
 - 则 $TF = 5 / 100 = 0.05$ 。
 - IDF (Inverse Document Frequency) 逆文档频率:
 - 衡量该词在整个语料库中的重要性。
 - 例: 'apple' 出现在10篇文档中的8篇中,
 - 则 $IDF = \ln(10 / 8) = 0.223$ 。
 - TF-IDF 计算公式:
 - $TF\text{-}IDF = TF \times IDF = 0.05 \times 0.223 = 0.01115$

Text Classification

- Typical supervised text classification contains two phases:

- Training: Train up the classification system based on sample data
- Prediction: Predict new data belongs to which class



Blueprint for text classification system.
Src: Dipanjan Text Analytics with Python, 2016

1.4 情感分析 (Sentiment Analysis)

- 情感分析：从文本中识别情绪/态度。
 - 粗粒度：正面 / 负面 / 中性
 - 细粒度：开心、愤怒、惊讶等。
- 工具：VADER (NLTK)
 - `SentimentIntensityAnalyzer()` → 生成情感得分。
 - 规则：
 - ◆ 分数 ≤ -0.05 → 负面
 - ◆ 分数 ≥ 0.05 → 正面
 - ◆ 其余为中性